

# Where Things Stand

Use an AI coding agent to autonomously convert ROCKE-3D (NASA GISS's Fortran GCM) to Python/JAX, and measure accuracy, CPU/GPU performance, conversion cost, and gotchas — validated against real production restart data (P2SAoM40).

**14 / 17**

## Modules faithfully ported

Validated directly against real Fortran. 3 remain placeholders — see slide 3.

**1e-9 to 1e-3**

## Accuracy vs. real Fortran

PBL + DRYCNV, same inputs, CPU and GPU — floating-point-level agreement.

**2.3–6.3×**

## GPU speedup vs. real Fortran

Measured, not estimated — first real GPU run, 2026-09-20 (kernel-level).

**Mixed on CPU**

## JAX is not a blanket CPU win

Fortran beats JAX-CPU on DRYCNV. The case for JAX here is GPU, not CPU.

The comparison that matters: Every number here is **JAX vs. the real production Fortran model.**

# Real Fortran vs. JAX — CPU and GPU

FULL PHYSICS CHAIN — THE HEADLINE RESULT

## Four stages, one real DTsrc step

PBL+radiation+surface+ground, P2SAoM40's real restart state. Radiation excluded (graybody stand-in) to keep every leg fair.

Log-scale bars.



4.2x GPU-vs-CPU on the optimized code (same A100, before/after fusing ~33 per-step JIT dispatches into one) -- clears the >2x target. Verification + a measurement-script bug this also fixed: next slide.

## What "62x" is actually measuring

Real Fortran SURFACE() does more physics than this JAX driver: area-weighted sub-tiling, full GHY land hydrology (not wired into JAX), and a real tridiagonal turbulence solve (JAX uses a shortcut).

So part of 62x is JAX computing faster -- part is JAX computing less physics for that scope. A faithful full port would likely see a smaller ratio.

## Kernel-level (mean of 100 calls)

Isolated routine, not the full pipeline.

13.4x slower

DRYCNV, JAX-CPU

2.3-6.3x faster

DRYCNV/PBL, JAX-GPU

# Four Stages, Verified — Not Just Measured

This project already found one measurement bug in this exact chain. Before shipping "62x faster," here's what was checked before believing it.

## Three independent checks

### 1. Did the fix touch the GPU number?

No. Dified it line-by-line: the GPU timing code (warm-up, block\_until\_ready before and after) is byte-for-byte unchanged. Only the mislabeled "CPU" section changed.

### 2. Does it reproduce?

Two independent runs of the same post-fusion code, same A100: 4.08 ms and 4.26 ms -- ~4% apart, normal jitter, not a fluke.

### 3. Does the magnitude make sense?

3,312 points, one fused dispatch, on an A100: low-single-digit ms is expected, not suspiciously fast. And 4.2x (not 40x) fits a grid too small to fully saturate the GPU -- the same real effect the old buggy number was chasing.

## The scope caveat -- read before quoting 62x

The numbers are real. What they compare isn't two implementations of identical work: real Fortran's SURFACE() does area-weighted multi-type sub-tiling, full land hydrology (GHY), and a real tridiagonal turbulence solve -- none of which the JAX full-chain driver currently does.

**A faithful, fully-featured port would do more work than today's simplified driver and would likely land at a smaller -- still real -- multiple.**

## Bottom line

17.94 ms → 4.26 ms is a real, reproducible **4.2x GPU speedup** on the physics this driver actually runs today -- clearing this project's own >2x target from real dispatch-fusion engineering, not a measurement artifact.

Treat it as a validated, scope-bounded win: real for what's ported today, not yet a claim about the full model.

# Recommendation & Open Items

## The remaining 3 modules — port or defer?

Why they're still placeholders: the original port deliberately left these three as documented stand-ins (e.g. LAKES' lkmix is a no-op) to reach interface-complete (17/17) first, prioritizing full validation of the columnar physics (PBL, DRYCNV) instead -- a scoping choice, not an oversight.

### SEAICE (core thermodynamics) — port next

Physically active in every P2SAoM40-class run, not optional. Dominates polar surface energy balance — exactly where the current 0.987/0.965 global correlation checks are least able to see a local error.

### ATURB (PBL-top-finding) — port next

Feeds surface-flux accuracy globally, not just at the poles. Also tests whether the vmap-over-grid-cells approach holds up on harder physics.

### LAKES (lkmix) — defer

Small fraction of Earth's surface — lower accuracy payoff for the effort. Revisit only if a lake-focused science need requires it.

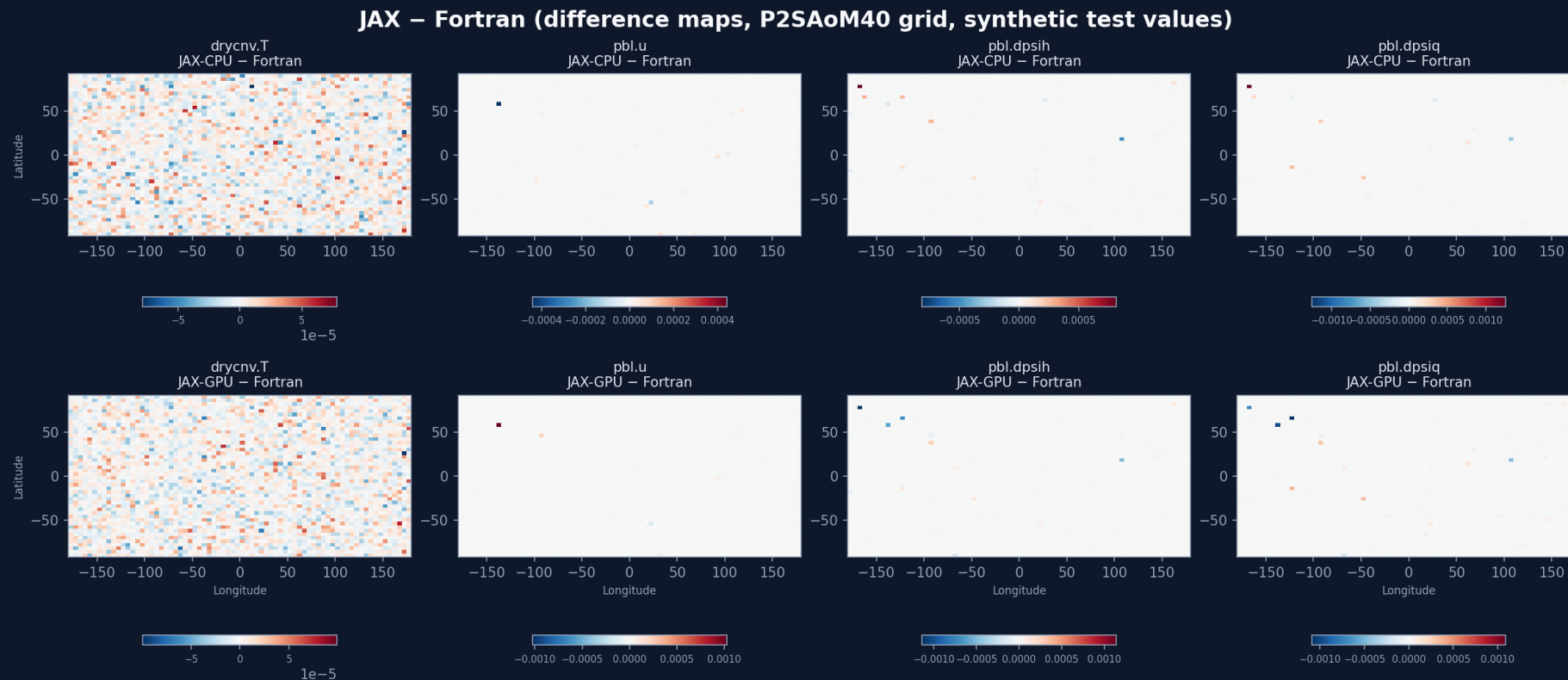
## Open items

- 1 Get the full physics-chain GPU speedup above 2x — **done**, 2026-09-22: fused ~33 per-step JIT dispatches into one, then found and fixed a bug where the "CPU" baseline was secretly timing the GPU. Real result: **4.2x** GPU speedup.
- 2 Fix the wind-speed convention bug in the shared FLUXES/SURFACE module files themselves — currently only worked around locally.
- 3 Port SEAICE + ATURB, then re-run the correlation check — expect it to improve or reveal exactly where it was masking a gap.

## OUTPUT MAPS

# Where JAX and Fortran Differ, Spatially

Difference maps only, from `visualize_p2saom40_kernel_maps.ipynb / outputs/p2saom40_kernel_*_diff_*.html`, on P2SAoM40's real 72x46 grid. Field values are synthetic test data, not real climate.



The bright outlier points in `pbl.u/dpsih/dpsiq` come from a few grid cells with near-zero Monin-Obukhov length -- both Fortran and JAX reproduce the same outliers, so these are still the same floating-point-level diffs reported in STATUS.md, not a bug.